

23 FPGA Implementation

A field-programmable gate array (FPGA) contains a configurable array of logic elements, memories, input/output pins, and wiring cells that can be programmed to perform arbitrary digital functions. To program an FPGA, design tools translate RTL into a *bitstream*, which configures the FPGA for particular logic functions, memory, and interfaces.

Semiconductor economic considers both *non-recurring engineering* costs (NRE) and *per-unit* cost. The NRE to create a mask set for an application-specific integrated circuit (ASIC) runs in the millions of dollars, and fabrication takes 3-12 weeks, then each chip costs under \$100 per unit (perhaps under \$1, for simple chips). In contrast, an FPGA evaluation board typically costs between \$100 and \$10,000, and a new bitstream can be created and downloaded in minutes to days depending on the size of the design. Therefore, FPGAs are suitable for rapid prototyping, while ASIC test chips are expensive and time-consuming to design and build. An FPGA can interact with external hardware in a way that is difficult to model in simulation. When a bug is found, the designer can generate and download a new bitstream to reprogram the FPGA.

Compared to ASICs, FPGAs tend to run at an order of magnitude lower clock frequency and cost two orders of magnitude more per gate. However, with parallelism and special-purpose logic, they can far exceed the performance and performance/watt of a general-purpose microprocessor. Therefore, FPGAs are suitable for simple or low-volume products where a general-purpose microprocessor is too slow. For example, for products selling thousands of units, the NRE of an ASIC can far exceed the per-unit cost. Therefore, FPGAs are especially suited to communication, scientific, medical, military, and aerospace applications that need flexibility and special-purpose logic, yet are produced in modest quantities.

FPGAs are also used in *emulation*, a logic verification technique where the design is mapped onto an FPGA so that large numbers of tests can be run on physical hardware at a much faster rate than is possible in software simulation. FPGAs can contain in-circuit debugging logic to improve observability and controllability during testing. Industrial SoC design teams use commercial FPGA-based emulation systems to replace hundreds of servers running verification tests for months or years.

23.1 FPGA Principles

FPGAs evolved from *programmable logic array* (PLA) chips, which were developed in the early 1980s to perform arbitrary sum-of-product functions. The hardware costs of PLAs grow quadratically with complexity, so FPGAs were developed in the mid-1980s as a more flexible way to implement larger systems.

23.1.1 Field-Programmable Gate Arrays

As Moore's Law continued to allow more transistors on a chip, the cost of logic arrays grew as the product of the number of inputs, making it inefficient to scale to large designs [Trimberger15]. Moreover, large numbers of product terms loaded down individual connections causing slow propagation delays. Consequently, a new solution was needed to provide better scalability [DeHon99].

Altera (now part of Intel), Xilinx (now part of AMD), and Actel (now Microsemi) were founded between 1983 and 1985 to develop chips with reprogrammable logic. Actel coined the term *field-programmable gate array* in 1988 [Trimberger15]. An FPGA has an array of programmable *logic elements* (LEs) connected by programmable interconnect. Each logic element typically contains a *look up table* (LUT) and a register. The LUT is a small memory that performs any function of a small number of inputs (typically 4-6). An FPGA performs complex logic functions by connecting outputs of one logic element to the inputs of another.

Figure 23.1 shows a generic FPGA architecture. The configurable logic elements (LEs) are surrounded by input/output elements (IOEs) that allow connections to the outside world. Configurable switches are shown in the hexagon within each LE intersection. FPGAs are programmed by downloading the contents of the LEs and the configuration of the switches. Most modern FPGAs use static RAM (SRAM) for configuration memory. Both the LUT contents and interconnect configurations are programmed by downloading a *bitstream file* into the configuration memory.

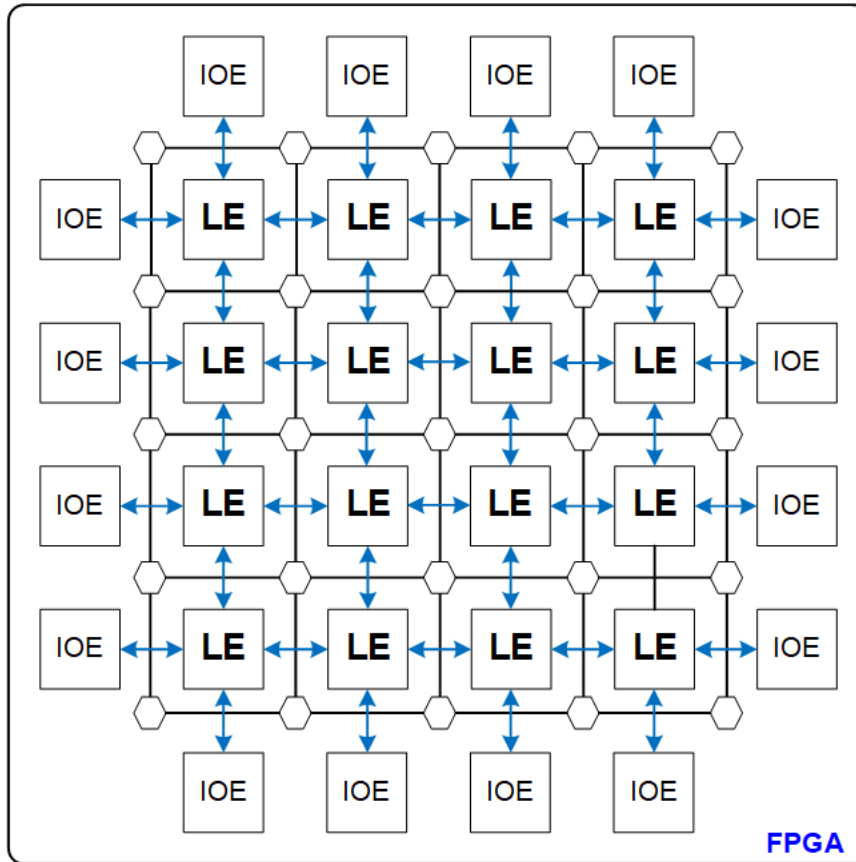


Figure 23.1 Generic array structure of an FPGA

Large FPGAs now contain millions of logic elements, along with other specialized blocks such as hundreds of megabits of SRAM, thousands of digital signal processing (DSP) units, high-speed serial I/O, multicore microprocessors, and clock generators. FPGAs also come with intellectual property (IP) libraries that include memory controllers, bus interfaces, and a variety of peripherals. They are sophisticated enough to implement complex systems-on-chip. Vendor-specific electronic design automation tools synthesize HDL code and map it onto a specific FPGA. In addition to design files, users provide constraint files that specify I/O pin assignments and target clock rate.

23.2 Wally FPGA Implementation

Wally boots Linux on an AMD (formerly Xilinx) FPGA. The FPGA implementation is useful for hands-on hardware designs and to diagnose logic bugs that are slow to reproduce in simulation. Specifically, the `fpga` configuration (a variation of `rv64gc`) of `wallypipelinedsoc` has been run on an Arty A7-100T board, an FPGA development board available from Digilent for \$299 in 2024. The board contains a Xilinx SC7A100TCSG324-1 FPGA, which is a mid-sized Artix-7 FPGA containing:

- 101,440 logic elements with 6-input lookup tables (LUTs)

- 5 Mib of RAM organized as 270 18Kb dual-ported blocks
- 240 digital signal processing (DSP) slices (25 x 18-bit multiplier, adder, and accumulator)
- 6 clock managers with *phase-locked loops* (PLLs)
- 210 user I/O pin

Table 23.1 lists the hardware required to build this FPGA implementation. Any micro B USB cable or SD Card will work.

Table 23.1 Required FPGA hardware

Item	Description	Vendor	Price
Arty A7	FPGA development board	digilent.com/shop/arty-a7-100t-artix-7-fpga-development-board/	\$299.00
Micro USB Cable	Power, programming, and communication	digilent.com/shop/usb-a-to-micro-b-cable/	\$4.99
PmodSD Board	Adapter board to connect to external flash card	digilent.com/reference/pmod/pmodsd/	\$9.99
SD Card	Non-volatile storage for operating system, programs, and data	digilent.com/shop/microsd-card-with-adapter/	\$12.99
Flash Card Reader	Read and write SD card from development PC	www.westerndigital.com/products/accessories/sandisk-imagemate-pro-usb-c-multi-card-reader-writer?sku=SDDR-A631-ANGNN	\$29.99

The `wallypipelinedsoc` module includes most of the hardware needed for a complete SoC but requires additional peripherals to implement a functional computer. Wally does not include a DRAM memory controller, a PLL to generate arbitrary clock frequencies, or an interface to non-volatile memory storage such as a *Secure Digital (SD) flash card*. The Wally FPGA implementation uses IP from AMD and open-source projects to fill these gaps and build a fully-functional general-purpose computer.

Figure 23.2 shows the block diagram of the Arty A7 board in light blue and the top-level Verilog module, `fpgatopArtyA7`, implemented on the Atrix-7 FPGA in medium blue. The board connects the FPGA to 256 MiB DDR3 memory, a 100 MHz clock, various I/O devices, buttons, LEDs, and breakout ports for expansion boards.

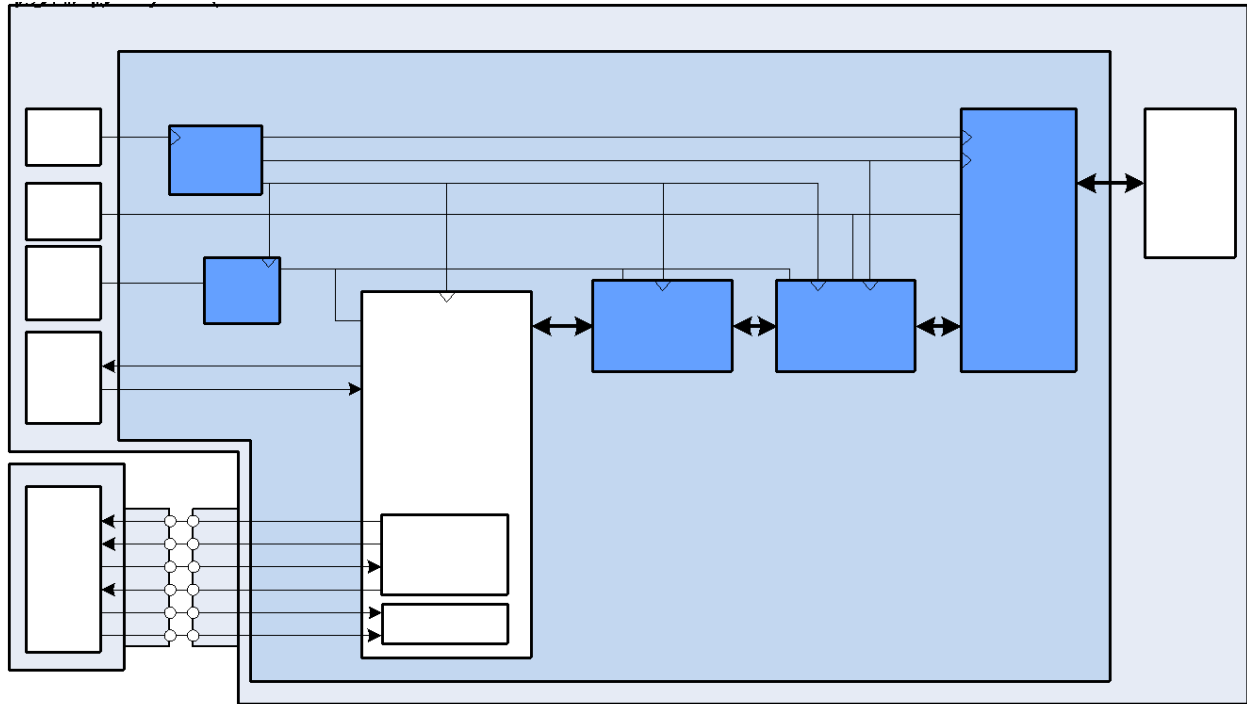


Figure 23.2 Arty A7 FPGA block diagram

`fpgatopArtyA7` instantiates the `wallypipelinedsoc` along with 5 IP blocks divided into three categories; clock and reset, bus converters, and memory controller. Table 23.2 describes the new AMD IP modules.

Table 23.2 AMD IP modules

Item	Description
<code>sysrst</code>	Synchronize reset to clock
<code>mmcm</code>	Clock generator
<code>ahbaxibridge</code>	Converts AHB to AXI4
<code>clkconverter</code>	Clock crossing domain between <code>CPUCLK</code> and 200 MHz
<code>memctrlddr3</code>	DDR3 memory controller

The *mixed-mode clock manager* (`mmcm`) is AMD IP that contains a PLL which takes in the 100 MHz on-board clock and generates three clocks: 167 MHz (for memory controller calibration), 200 MHz (for the DDR memory interface), and 20 MHz (`CPUCLK` for the Wally core). The system reset module (`sysrst`) takes in an asynchronous reset button and produces `reset_periph` synchronized to the 20 MHz `CPUCLK`. The DDR3 DRAM controller (`memctrlddr3`) takes in the `resetrn` button and synchronizes it to the 200 MHz clock. The board has two reset buttons: `btn0` (`reset_periph`) resets both Wally and the memory controller, while `resetrn` only resets the memory controller.

The AMD DRAM controller (`memctrlddr3`) implements the AXI4 subordinate protocol that converts bus read and writes into DDR3 DRAM read and writes (see Section 9.2XREF).

`memctrlddr3` also handles the nuances of DRAM management including I/O timing, skew compensation, periodically refreshing each memory address, and opening and closing memory banks. The full scope of memory controller design is its own textbook. The DRAM controller requires a 200 MHz logic clock and a 167 MHz I/O timing skew compensation clock.

<SIDEBAR>

Wally supports higher performance FPGA boards such as the VCU108 and VCU118 which have 2 GiB DDR4 memory and use a different MIG (memory interface generator) architecture.

</END>

The `wallypipelinedsoc` external bus connection is AHB and runs at the 20 MHz core clock frequency, while the DDR3 memory controller (`memctrlddr3`) uses AXI4 and runs at 200 MHz. Glue logic converts between the two bus protocols and a FIFO clock converter (`clkconverter`) buffers the two clock domains. AMD provides IP for both. The Xilinx-AHBLite-AXI bridge (`ahbaxibridge`) translates the AHB bus into AXI4 transactions. The Xilinx AXI clock converter (`clkconverter`) is an asynchronous first in first out (FIFO) queue to facilitate communication between the AHB and AXI4 clock domains.

Wally's uses a second instance of the SPI peripheral, Section 20.5XREF to implement the SD Card Controller. Section 23.2.1 describes additional details specific to the SPI based SD Card implementation.

Figure 23.3 highlights the important components of the Arty A7 board. The Artix-7 FPGA is located in the center with the DDR3 memory off to its right. The USB cable provides both power and communications. USB connects two UART controllers: one for programing and debugging the FPGA (see Section 23.1.1.3), and the second for connecting to Wally's UART (see Section 23.1.1.4). The PmodSD board connects via the right-most Pmod port D. `resetn` resets all modules on the FPGA. The second reset `bnt0` only resets `wallypipelinedsoc`.

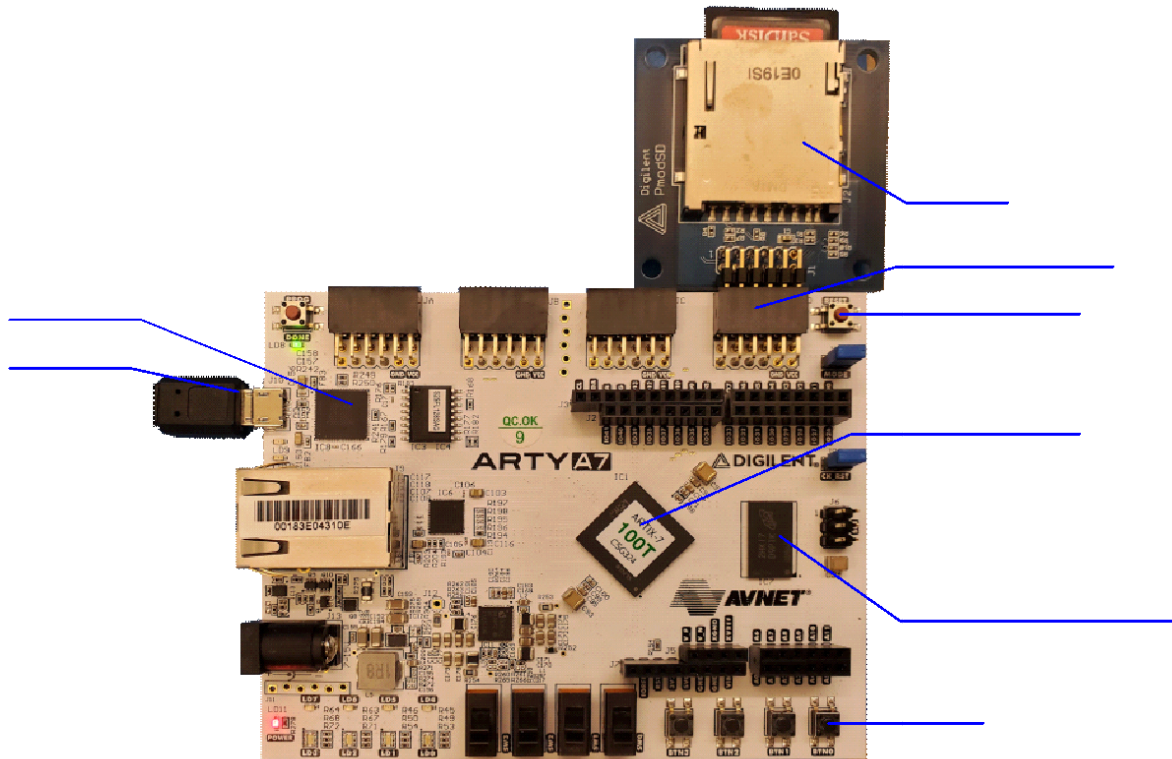


Figure 23.3 Arty A7 FPGA and PmodSD DRAM expansion card

23.2.1 SD Card

The SD card communicates using the 1-bit SPI interface. With a clock frequency of f_c , the bandwidth is $(f_c \text{ transactions/s}) \times (1 \text{ bits/transactions}) / (8 \text{ bits/byte}) = f_c/8 \text{ bytes/s}$. The maximum frequency of 25 MHz delivers 3.125 MB/s. While relatively slow, SPI has multiple advantages because it not require a costly license and the Linux kernel contains built in drivers.

<SIDEBAR>

SD Cards have a second faster interface, *SDIO*, which runs at 50MHz with 4 bit/transaction delivering 25 MB/s. However, the interface requires a costly license.

</END>

SPI does not have a way to indicate when a card is inserted or if the card is write-protected. A pin from the card socket called *card detect* (CD) drives a GPIO pin to interrupt the CPU when a card is inserted. Similarly, another pin called *write protect* (WP) drives another GPIO pin to prevent the CPU from writing the SD card if high.

23.1.1 Arty A7 Setup

This section describes how to build a Wally SoC targeted to an Arty A7 development board, which is then capable of booting Buildroot Linux with an interactive command-line console. As described in detail in the remainder of this section, the four main steps are:

1. Build the SD flash card to hold a Buildroot Linux image
2. Synthesize Wally and place and route on FPGA
3. Program the FPGA
4. Connect a UART terminal interface

<SIDEBAR>

Buildroot is an automated system for cross-compiling Linux to embedded platforms described in Section 22.2XREF.

</END>

23.1.1.1 Build the SD Flash Card

The bootloader, OS, and user programs are stored on an SD flash card. The bootloader copies the OS from the flash card into main memory and then begins executing the operating system. Specifically, Wally's Zero-Stage Bootloader (ZSBL) first reads the *GPT partition* information and copies the device tree and OpenSBI from the flash card into main memory. The *GUID Partition Table* (GPT) defines the location and size of partitions on the flash card. Section 23.1.3 describes the bootloader in more depth. OpenSBI then uses the ZSBL to read the Linux kernel from the flash card into main memory and boot into Buildroot. To generate the flash card image, first we need to create the flattened device tree, then partition the flash card, and copy all the relevant files.

<SIDEBAR>

A device tree is a human readable description of a hardware configuration that defines clock speeds, memory mapped I/O addresses, main memory location and size, and many other features. See Section 22.1.3XREF for more details.

</END>

Section 22.2.9XREF defines the device tree for virtual hardware emulation using QEMU. The Arty A7 uses the `cvw/linux/devicetree/wally-artya7.dts` device tree, which specifies a 20 MHz clock and 256 MiB main memory. This device tree uses the same addresses and interrupt IRQs from Figure 22.2XREF but adds the following section for the flash card.

```
refclk: refclk {
    #clock-cells = <0>;
    compatible = "fixed-clock";
    clock-frequency = <20000000>;
    clock-output-names = "xtal";
};

gpio0: gpio@10060000 {
    compatible = "sifive,gpio0";
    interrupt-parent = <0x03>;
```



```

        interrupts = <3>;
        reg = <0x00 0x10060000 0x00 0x1000>;
        reg-names = "control";
        gpio-controller;
        #gpio-cells = <2>;
        interrupt-controller;
        #interrupt-cells = <2>;
    };
    spi@13000 {
        compatible = "sifive,spi0";
        interrupt-parent = <0x03>;
        interrupts = <0x14>;
        reg = <0x0 0x13000 0x0 0x1000>;
        reg-names = "control";
        clocks = <&refclk>;

        #address-cells = <1>;
        #size-cells = <0>;
        mmc@0 {
            compatible = "mmc-spi-slot";
            reg = <0>;
            spi-max-frequency = <1000000>;
            voltage-ranges = <3300 3300>;
            disable-wp;
            // gpios = <&gpio0 6 1>;
        };
    };
};

```

refclk defines a clock required by the spi device. **gpio0** defines the two GPIO pins for CD and WP while **spi** defines the spi device for the flash card. Finally, we change **bootargs** to set the UART baud rate to 115200.

```
bootargs = "root=/dev/vda ro console=ttyS0,115200";
```

Then compile Buildroot with the options from Section 22.2.1XREF.

Next, insert a flash card into the computer and copy the memory image to the card. The flash card will appear as an `sd{a, b, c, ...}` file in the `/dev` directory. The easiest way to determine which one it is, is to `ls /dev/sd*` before inserting the card and then `ls /dev/sd*` after inserting the card. The new device is the flash card.

Then run this script with the `/dev/sd<letter>` from above to write the flash card. This script creates a GPT (GUID partition table) and then 4 partitions. Partition 1 contains the device tree, partition 2 OpenSBI, partition 3 Buildroot, and partition 4 a mountable file ext4 file system.

```
$ cd $WALLY/linux/sdcard
$ ./flash-sd.sh -d wally-artya7.dtb /dev/sd<letter>
```

23.1.1.2 Synthesize Wally and Place and Route on FPGA

Vivado is an electronic design automation tool that translates RTL into a hardware configuration bitstream to program an AMD FPGA. To install Vivado, first download the latest version from

Xilinx’s website, www.xilinx.com/support/download.html. At the time of writing this was version 2023.1. Install on Linux using:

```
$ cd ~/Downloads
$ chmod +x Xilinx_Unified_2023.1_0507_1903_Lin64.bin
$ sudo ./Xilinx_Unified_2023.1_0507_1903_Lin64.bin
```

<SIDEBAR>

Vivado is over 90GiB.

</SIDEBAR>

Then provide your AMD login and select install Vivado. For the free version select “Vivado ML Standard” or for paid version “Vivado ML Enterprise.” Click next and agree to everything. Install it to your desired directory. Once installed, click “OK” to finish.

The license manager will be active after installation. If you are using the free version, follow the instructions in the install wizard. If you have a site license, close the license manager, and edit your `~/.bashrc` file to append a line pointing to the license:

```
$ export XILINXD_LICENSE_FILE=<port number>@<server name>
```

Once the installation finishes, copy the board support files from the vivado-boards submodule into the install directory.

```
$ sudo cp -r cvw/addins/vivado-boards/new/board_files <install path>/<version>/data/boards/
```

The next step is to synthesize the Wally RTL into a bitstream to program the FPGA using Vivado. On a fast computer, synthesis, place and route, and bitstream generation takes around 20 minutes. The process is automated using Makefiles to build the various AMD IP modules from Section 23.2 and Wally. The **Makefile** also optionally includes the *In-circuit Logic Analyzer* (ILA), which functions similar to a benchtop logic analyzer but probes internal FPGA wires. Section 23.1.1.6 details how to use the ILA.

The Makefile supports multiple development boards which can be specified by name at the command line. The Arty A7 is the default and will be built if another target is not provided. To synthesize Wally to the Arty A7 FPGA development board:

```
$ cd $WALLY/fpga/generator
$ make
```

When synthesis completes, exit the Vivado prompt with:

```
$ exit
```

The **Makefile** begins by building the Vivado IP using the *.tcl scripts. For example, `_ddr3-ArtyA7.tcl` builds the memory controller IP for the Arty A7 board. Then it copies all the Wally source code from `cvw/src/` into `cvw/fpga/src/CopiedFiles_do_not_add_to_repo` and

adds directives to preserve signal names during synthesis for debugging. The Makefile then calls the wally.tcl script to begin synthesis, place and route, and bitstream generation. wally.tcl uses constraint files defined in cvw/fpga/constraints/. The Arty A7 board uses constraints-ArtyA7.xdc and small-debug.xdc.

23.1.1.3 Program the FPGA

Now you are ready to configure the FPGA on the Arty A7 board as the Wally SoC. To do so, first run Vivado in GUI mode with:

```
$ vivado &
```

Click on “Open Project”, select “WallyFPGA.xpr”, and click “Ok”, Figure 23.4.

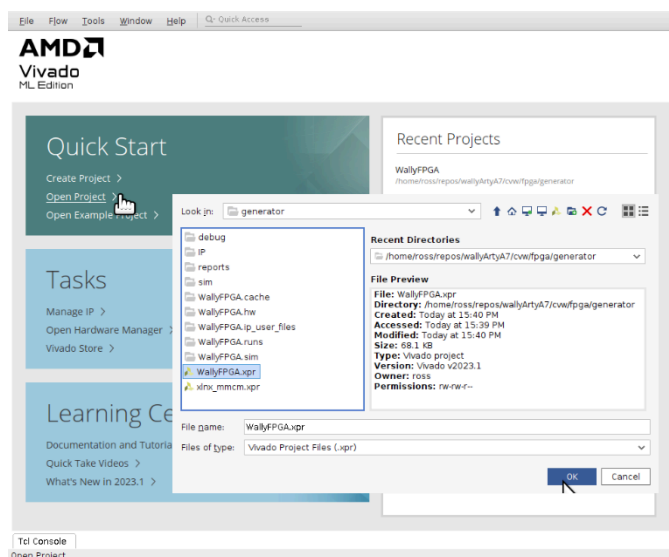


Figure 23.4 Open Wally Vivado project

On the lefthand side of the screen is the “Project Manager”. At the bottom, click on “Open Hardware Manager”. Before programming the FPGA, remove the flash card from the computer and insert it into the Digilent PmodSD expansion board. Insert the expansion board into the Arty A7 board’s right most Pmod socket (called port D), as shown in Figure 23.3. Plug the USB cable into the FPGA and computer and click “Open Target”, Figure 23.5 and then “Auto connect”.

<SIDEBAR>

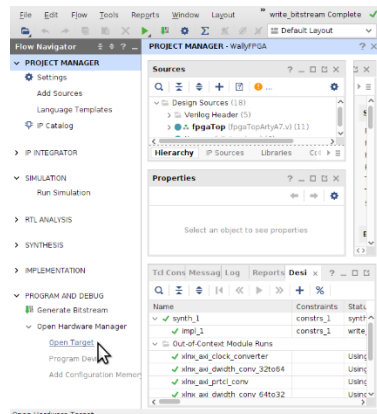


Figure 23.5 Open Hardware Manager and Open Target

</END>

Now program the FPGA by clicking “Program Device” (see Figure 23.6 and then clicking the popup FPGA part number. After a few seconds the FPGA will be configured as the Wally SoC using the generated bitstream, and it will begin booting Linux.

<SIDEBAR>

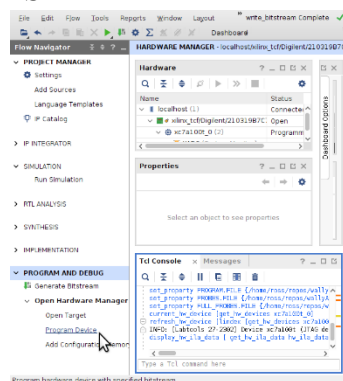


Figure 23.6 Program Device

</END>

<SIDEBAR>

Vivado uses JTAG over USB to program the FPGA's bit stream. Occasionally, Vivado incorrectly sets the JTAG clock frequency too high. The ILA specifies the JTAG clock should be no more than 2.5 times as fast as the debugger's clock frequency. Wally uses the same clock for the debugger as the main 20MHz clock. If you cannot establish communication with the FPGA, set the JTAG clock to 7.5MHz, as shown in Figure 23.7.

1. Select "Program and Debug" in the "Project Navigator".
2. To the right of "Flow Manager" is "Hardware". Expand "localhost" and select "xilinx_tcf/Digilent/21031..." The specific number may vary.
3. Go to the box below, "Hardware Target Properties".
4. On the last line select "Properties".
5. Expand "Param" and change "Frequency" to 7500000.

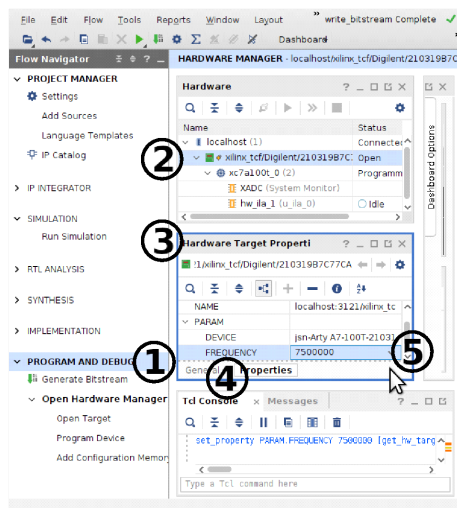


Figure 23.7 JTAG settings

</END>

23.1.1.4 Serial Terminal Over UART

As OpenSBI and Linux boot they display information over the UART serial console and eventually display a login prompt where the user can interactively issue commands in real time. This section describes how to set up the serial console, change baud rates and device permissions, and run programs on Wally.

The Arty A7 board's USB connection provides two devices, one for programming and debugging and the second as a USB-to-UART adaptor. A UART (universal asynchronous receiver transceiver, see Section 19.4XREF), is a full-duplex clockless serial communication standard which appears as a tty serial port to a GNU/Linux operating system. Many different programs can interact with a tty serial port, but for this example we will use the program *screen*.

First, set the file permissions for the serial device. The UART device will appear as a `/dev/ttyUSB<number>`. Check the group permissions. The examples below assume the device number is `ttyUSB0`, but replace “0” with your device number.

<SIDEBAR>

If your computer has multiple USB-based tty devices, the device number may be something other than `ttyUSB0`.

</END>

```
$ ls /dev/ttyUSB0 -l
crw-rw---- 1 root dialout 188, 2 Jul 17 14:45 /dev/ttyUSB0
```

The fourth column is the group, and it should be `dialout`. If it is something else, change the group with:

```
$ sudo chgrp dialout /dev/ttyUSB0
```

Next, verify your user is part of the `dialout` group with (assumes username is `wally`):

```
$ groups
wally adm dialout cdrom sudo dip plugdev lpadmin lxd sambashare
```

If your user is not part of `dialout`, add it with:

```
$ usermod -a -G dialout <your username>
```

Now, use the following to connect to the Arty A7 over UART:

```
$ screen /dev/ttyUSB<number> 115200
```

115200 is the baud rate, which was specified in the device tree. After about 40 seconds OpenSBI prints a banner, see Figure 23.8. Over the next minute Linux outputs boot messages and prompts the user for root login. The messages are similar to Figure 22.3XREF except that the timestamps are different in hardware versus simulation.

OpenSBI v1.3

```

  /_/_/  /_/_/_/_/_/
  |||_/_/_/_/_/_/|||
  |||_/_/_/_/_/_/|||
  |||_/_/_/_/_/_/|||
  \_/_/_/_/_/_/_/_/_/
  ||
  ||
  ||
```

```
Platform Name       : wally-virt,qemu
Platform Features   : medeleg
Platform HART Count : 1
Platform IPI Device : aclint-mswi
Platform Timer Device : aclint-mtimer @ 200000000Hz
Platform Console Device : uart8250
Platform HSM Device : ---
Platform PMU Device : ---
Platform Reboot Device : ---
Platform Shutdown Device : ---
Platform Suspend Device : ---
Platform CPPC Device : ---
Firmware Base       : 0x80000000
Firmware Size       : 194 KB
```

```

Firmware RW Offset : 0x20000
Firmware RW Size : 66 KB
Firmware Heap Offset : 0x28000
Firmware Heap Size : 34 KB (total), 2 KB (reserved), 9 KB (used), 22 KB (free)
Firmware Scratch Size : 4096 B (total), 760 B (used), 3336 B (free)
Runtime SBI Version : 1.0

Domain0 Name : root
Domain0 Boot HART : 0
Domain0 HARTs : 0*
Domain0 Region00 : 0x0000000002000000-0x000000000200ffff M: (I,R,W) S/U: ()
Domain0 Region01 : 0x0000000008000000-0x0000000008001ffff M: (R,X) S/U: ()
Domain0 Region02 : 0x00000000080020000-0x0000000008003ffff M: (R,W) S/U: ()
Domain0 Region03 : 0x0000000000000000-0xffffffff M: (R,W,X) S/U: (R,W,X)
Domain0 Next Address : 0x0000000008020000
Domain0 Next Arg1 : 0x0000000008220000
Domain0 Next Mode : S-mode
Domain0 SysReset : yes
Domain0 SysSuspend : yes

Boot HART ID : 0
Boot HART Domain : root
Boot HART Priv Version : v1.12
Boot HART Base ISA : rv64imafdc
Boot HART ISA Extensions : time,sstc
Boot HART PMP Count : 16
Boot HART PMP Granularity : 4
Boot HART PMP Address Bits: 54
Boot HART MHPM Count : 0
Boot HART MIDELEG : 0x0000000000000222
Boot HART MEDELEG : 0x000000000000b108
[ 0.000000] Linux version 6.6.0 (rose@coco) (riscv64-buildroot-linux-gnu-gcc.br_real (Buildroot 2023.05.3-dirty) 12.3.0, GNU ld (GNU Binutils) 2.38) #2 Sun Nov 19 11:57:27
CST 2023
[ 0.000000] Machine model: wally-virt,qemu
[ 0.000000] SBI specification v1.0 detected
[ 0.000000] SBI implementation ID=0x1 Version=0x10003
[ 0.000000] SBI TIME extension detected
[ 0.000000] SBI IPI extension detected
[ 0.000000] SBI RFENCE extension detected
[ 0.000000] efi: UEFI not found.
[ 0.000000] OF: reserved mem: 0x0000000080000000..0x000000008001ffff (128 KiB) nomap non-reusable mmode_resv0@80000000
[ 0.000000] OF: reserved mem: 0x0000000080020000..0x000000008003ffff (128 KiB) nomap non-reusable mmode_resv1@80020000
[ 0.000000] Zone ranges:
[ 0.000000] DMA32 [mem 0x0000000080000000-0x000000008ffffff]
[ 0.000000] Normal empty
[ 0.000000] Movable zone start for each node
[ 0.000000] Early memory node ranges
[ 0.000000] node 0: [mem 0x0000000080000000-0x000000008003ffff]
[ 0.000000] node 0: [mem 0x0000000080040000-0x000000008ffffff]
[ 0.000000] Initmem setup node 0 [mem 0x0000000080000000-0x000000008ffffff]
[ 0.000000] Falling back to deprecated "riscv,isa"
[ 0.000000] riscv: base ISA extensions acdfim
[ 0.000000] riscv: ELF capabilities acdfim
[ 0.000000] Kernel command line: root=/dev/vda ro console=ttyS0,115200
[ 0.000000] Dentry cache hash table entries: 32768 (order: 6, 262144 bytes, linear)
[ 0.000000] Inode-cache hash table entries: 16384 (order: 5, 131072 bytes, linear)
[ 0.000000] Built 1 zonelists, mobility grouping on. Total pages: 64640
[ 0.000000] mem auto-init: stack:off, heap alloc:off, heap free:off
[ 0.000000] Virtual kernel memory layout:
[ 0.000000] fixmap : 0xffff8d7f00000000 - 0xffff8d7fff000000 (6144 kB)
[ 0.000000] pci io : 0xffff8d7ff0000000 - 0xffff8d8000000000 ( 16 MB)
[ 0.000000] vmemmap : 0xffff8d8000000000 - 0xffff8f8000000000 (2048 GB)
[ 0.000000] vmalloc : 0xffff8f8000000000 - 0xffffaf8000000000 ( 32 TB)
[ 0.000000] modules : 0xffffffff01ee5000 - 0xffffffff80000000 (2017 MB)
[ 0.000000] lowmem : 0xffffaf8000000000 - 0xffffaf8010000000 ( 256 MB)
[ 0.000000] kernel : 0xfffffff800000000 - 0xffffffffff000000 (2047 MB)
[ 0.000000] Memory: 198184K/262144K available (2527K kernel code, 2970K rdata, 2048K rodata, 22277K init, 242K bss, 63960K reserved, 0K cma-reserved)
[ 0.000000] SLUB: HWalign=64, Order=0-3, MinObjects=0, CPUs=1, Nodes=1
[ 0.000000] NR_IRQS: 64, nr_irqs: 64, preallocated irq: 0
[ 0.000000] riscv-intc: 64 local interrupts mapped
[ 0.000000] plic: plic@c000000: mapped 53 interrupts with 1 handlers for 2 contexts.
[ 0.000000] clocksource: riscv_clocksource: mask: 0xffffffff max_cycles: 0x49cd42e20, max_idle_ns: 440795202120 ns
[ 0.000444] sched_clock: 64 bits at 20MHz, resolution 50ns, wraps every 4398046511100ns
[ 0.007921] Console: colour dummy device 80x25
[ 0.010617] Calibrating delay loop (skipped), value calculated using timer frequency.. 40.00 BogoMIPS (lpj=80000)
[ 0.011472] pid_max: default: 32768 minimum: 301
[ 0.016652] Mount-cache hash table entries: 512 (order: 0, 4096 bytes, linear)
[ 0.017681] Mountpoint-cache hash table entries: 512 (order: 0, 4096 bytes, linear)
[ 0.117319] riscv: ELF compat mode unsupported
[ 0.119186] ASID allocator using 16 bits (65536 entries)
[ 0.131938] EFI services will not be available.
[ 0.154531] devtmpfs: initialized
[ 0.229558] clocksource: jiffies: mask: 0xffffffff max_cycles: 0xffffffff, max_idle_ns: 7645041785100000 ns
[ 0.231459] futex hash table entries: 256 (order: 1, 12288 bytes, linear)
[ 0.299644] cpu0: Ratio of byte access time to unaligned word access is 7.66, unaligned accesses are fast
[ 0.448336] clocksource: Switched to clocksource riscv_clocksource
[ 1.478560] workingset: timestamp_bits=62 max_order=16 bucket_order=0

```

```
[ 1.523903] io scheduler mq-deadline registered
[ 1.525224] io scheduler kyber registered
[ 1.527022] io scheduler bfq registered
[ 15.005563] Serial: 8250/16550 driver, 1 ports, IRQ sharing disabled
[ 15.145231] printk: console [ttyS0] disabled
[ 15.181518] 10000000.uart: ttyS0 at MMIO 0x10000000 (irq = 2, base_baud = 1250000) is a 16550A
[ 15.182922] printk: console [ttyS0] enabled
[ 16.663950] debug_vm_pgtable: [debug_vm_pgtable    ]: Validating architecture page table helpers
[ 17.066015] clk: Disabling unused clocks
[ 111.526296] Unpacking initramfs...
[ 111.531943] Initramfs unpacking failed: invalid magic at start of compressed archive
[ 113.847030] Freeing initrd memory: 26892K
[ 115.802062] Freeing unused kernel image (initmem) memory: 22276K
[ 115.812837] Run /init as init process
init started: BusyBox v1.36.1 (2023-11-19 11:51:43 CST)
Starting syslogd: OK
Starting klogd: OK
Running sysctl: OK
Saving 256 bits of non-creditable seed for next boot
Starting network: /etc/init.d/S40network: line 12: /sbin/ifup: not found
FAIL
```

BusyBox v1.36.1 (2023-11-19 11:51:43 CST) built-in shell (ash)
Enter 'help' for a list of built-in commands.

Hello this ~/profile is meant to demonstrate running some basic commands on Wally.

I am root

And I am on WallyHostname

Created myFile.txt and moved it to myDir. It contains:

Hello World!

And farewell!

Created myScript.sh. Running it yields:

Hello this is another example script

Created symLinkToMyScript.sh. Running it yields:

Hello this is another example script

Now let's remove all these example files and scripts

Here is disk usage:

Filesystem	Size	Used	Available	Use%	Mounted on
devtmpfs	96.8M	0	96.8M	0%	/dev
tmpfs	120.8M	0	120.8M	0%	/dev/shm
tmpfs	120.8M	0	120.8M	0%	/tmp
tmpfs	120.8M	8.0K	120.8M	0%	/run

And here are the current processes:

PID	USER	VSZ	STAT	COMMAND
1	root	2208	S	init
2	root	0	SW	[kthreadd]
3	root	0	SW	[pool_workqueue_]
4	root	0	IW<	[kworker/R-slab_]
5	root	0	IW	[kworker/0:0-eve]
6	root	0	IW<	[kworker/0:0H]
7	root	0	IW	[kworker/u2:0-ev]
8	root	0	IW<	[kworker/R-mm_pe]
9	root	0	SW	[ksoftirqd/0]
10	root	0	SW	[kdevtmpfs]
11	root	0	SW	[khungtaskd]
12	root	0	SW	[oom_reaper]
13	root	0	IW<	[kworker/R-write]
14	root	0	SW	[kcompactd0]
15	root	0	IW<	[kworker/R-kbloc]
16	root	0	IW	[kworker/0:1-pm]
17	root	0	SW	[kswapd0]
18	root	0	IW	[kworker/u2:1-ev]
19	root	0	IW	[kworker/u2:2-ev]
40	root	2208	S	/sbin/klogd -n
54	root	2208	S	~/bin/sh
70	root	2208	R	ps

And finally a login prompt.

WallyHostname login:

Figure 23.8 Linux boot messages on the Arty A7

The password is **root**. Once logged in, the terminal behaves just like a conventional Linux system. It is possible to run text editors and benchmarks, write new programs, and save files to the flash card. Run the CoreMark benchmark with the following:

```
$ coremark
```

2K performance run parameters for coremark.


```

CoreMark Size : 666
Total ticks : 14419
Total time (secs): 14.419000
Iterations/Sec : 41.611762
Iterations : 600
Compiler version : GCC12.3.0
Compiler flags : -D_LARGEFILE_SOURCE -D_LARGEFILE64_SOURCE -D_FILE_OFFSET_BITS=64 -Os -g3 -lrt
Memory location : Please put data memory location here
                  (e.g. code in flash, data on heap etc)
seedcrc : 0xe9f5
[0]crclist : 0xe714
[0]crcmatrix : 0x1fd7
[0]crcstate : 0x8e3a
[0]crcfinal : 0xbd59
Correct operation validated. See readme.txt for run and reporting rules.
CoreMark 1.0 : 41.611762 / GCC12.3.0 -D_LARGEFILE_SOURCE -D_LARGEFILE64_SOURCE -D_FILE_OFFSET_BITS=64 -Os -g3 -lrt / Heap

```

Another interesting benchmark is *ramspeed*, which measures memory bandwidth for various size blocks. Notice how the bandwidth is higher for blocks which fit into the 16KiB data cache.

```
$ time ramspeed -b 2 -m 1 -g 1
```

RAMspeed (GENERIC) v2.6.0 by Rhett M. Hollander and Paul V. Bolotoff, 2002-09

1Gb per pass mode

```

INTEGER & READING    1 Kb block: 131.87 MB/s
INTEGER & READING    2 Kb block: 134.91 MB/s
INTEGER & READING    4 Kb block: 135.49 MB/s
INTEGER & READING    8 Kb block: 131.78 MB/s
INTEGER & READING   16 Kb block: 115.55 MB/s
INTEGER & READING   32 Kb block: 27.38 MB/s
INTEGER & READING   64 Kb block: 27.38 MB/s
INTEGER & READING  128 Kb block: 27.12 MB/s
INTEGER & READING  256 Kb block: 27.20 MB/s
INTEGER & READING  512 Kb block: 27.19 MB/s
INTEGER & READING 1024 Kb block: 27.15 MB/s
INTEGER & READING 2048 Kb block: 27.19 MB/s
INTEGER & READING 4096 Kb block: 27.15 MB/s
INTEGER & READING 8192 Kb block: 27.17 MB/s
INTEGER & READING 16384 Kb block: 27.14 MB/s
INTEGER & READING 32768 Kb block: 27.20 MB/s
ramspeed -b 2 -g 1 -m 1 477.60s user 12.93s system 99% cpu 8:11.31 total

```

23.1.1.5 AHB Burst Performance

Section 12.2.1XREF introduced the LSU with a D\$ that uses AHB burst cache line fetches. Burst mode dramatically improves memory bandwidth and, subsequently, Linux boot time. To test and measure Wally's performance without AHB burst edit \$WALLY/src/ebu/buscachefsm.sv by setting `define BURST_EN 0. Now run ramspeed again:

```
$ time ramspeed -b 2 -m 1 -g 1
```

RAMspeed (GENERIC) v2.6.0 by Rhett M. Hollander and Paul V. Bolotoff, 2002-09

1Gb per pass mode

```

INTEGER & READING    1 Kb block: 108.99 MB/s
INTEGER & READING    2 Kb block: 110.97 MB/s
INTEGER & READING    4 Kb block: 104.27 MB/s
INTEGER & READING    8 Kb block: 80.59 MB/s

```

```

INTEGER & READING    16 Kb block: 3.42 MB/s
INTEGER & READING    32 Kb block: 3.10 MB/s
INTEGER & READING    64 Kb block: 3.09 MB/s
INTEGER & READING    128 Kb block: 3.07 MB/s
INTEGER & READING    256 Kb block: 3.09 MB/s
INTEGER & READING    512 Kb block: 3.11 MB/s
INTEGER & READING    1024 Kb block: 3.13 MB/s
ramspeed -b 2 -g 1 -m 1 2418.09s user 8.75s system 99% cpu 40:43.10 total

```

The program takes 7.8 times longer to execute and the data strides ($\geq 32\text{KiB}$) which don't fit into the cache show a dramatic drop in bandwidth, about $1/8^{\text{th}}$ of the original. This configuration has 8 words per cache line, so one cache line fetch or writeback burst becomes 8 single transactions.

With burst, the Arty A7 boots Linux in 2 minutes and 17 seconds, but without burst it takes 7 minutes and 43 seconds. The reason burst is so effective is that DRAM memories are optimized for large transfers but require many cycles of overhead to initiate a transaction. Burst amortizes that cost for all the words in a cache line, thus reducing the average memory latency. It is worth revisiting Amdahl's Law which states: "Make the common case fast." Typically, we assume a program's working set will fit into the L1 cache. However, this assumption fails for the majority of ramspeed's execution and, thus, results in overall poor performance. For comparison CoreMark scores 37.5 without burst, just 22% slower.

23.1.1.6 In-Circuit Logic Analyzer

After programming the FPGA with the Wally SoC, a new window to the right of the "Hardware" panel called "hw_ila_1" displays a logic analyzer interface. Press the double arrow button (Figure 23.9) to capture a sample of the processor state. Table 23.3 lists the signals the ILA captures in the Memory stage. Below the "Hardware" box is a trigger configuration. Click the plus icon to add a signal to the trigger configuration. For example, select PCM and set the hex value to 0x0000_0000_0000_1186. Then press the triangle play button to wait until the trigger occurs.

Table 23.3 ILA probes

Signal Name	Width
PCM	XLEN
InstrValidM	1
InstrM	32
TrapM	1
IEUAdrM	XLEN
MemRWM	2

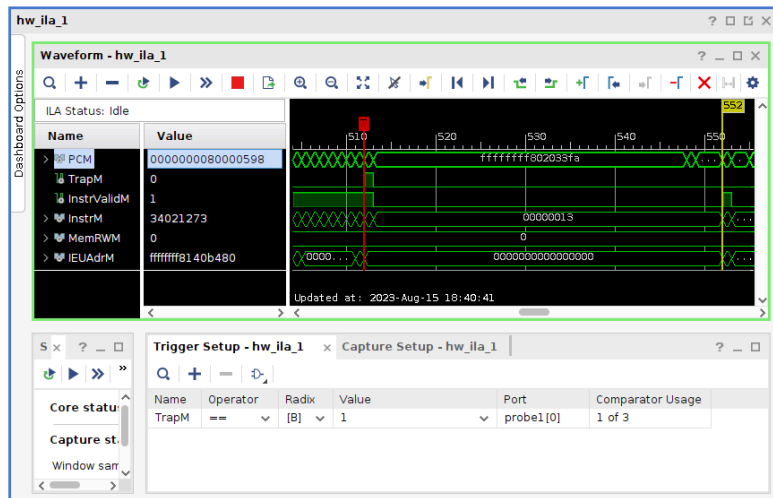


Figure 23.10 Example OpenSBI TrapM

The ILA uses BRAM, LUTs, and flops. The Arty A7 supports a small number of useful signals, but it is sometimes necessary to add other signals. This requires two steps. First the logic must be annotated with `(* mark_debug = "true" *)` in the source code so the optimizer knows not to remove the net from the netlist. Since adding `mark_debug` throughout the Wally RTL is distracting, we instead opt to copy to source code into `$WALLY/fpga/src/CopiedFiles_do_not_add_to_repo/`. Then the FPGA Makefile calls `insert_debug_comment.sh`, which uses `$WALLY/fpga/constraints/marked_debug.txt` to add the `mark_debug` directives to the copied files. Second, the ILA is configured to probe the net using the following TCL commands.

```
create_debug_port u_ila_0 probe
set_property port_width 1 [get_debug_ports u_ila_0/probe5]
set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe5]
connect_debug_port u_ila_0/probe5 [get_nets [list wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/LSUHREADY ]]
```

These are most easily added to a debugger ILA file in the `cvw/fpga/constraints` directory. The Arty A7 uses `small-debug.xdc`. The probe<number> needs to be incremented for each additional port, and the port width must exactly match the number of bits in the connected signal. A multiple-bit bus must have each bit explicitly enumerated. See `small-debug.xdc` for an example.

The `proberange` Python script in the `cvw/fpga/constraints` directory automates the laborious task of generating these debug scripts. `proberange` takes two arguments: a file with a list of signals, and an output file. `proberange` generates the four ILA probe commands. The first file is formatted as `<signal_name width probe_number>`. The signal name must match the exact path in the instance hierarchy.

For example, to add the LSUHADDR bus as probe number 6 we do the following.

1. Edit `marked_debug.txt` to include `lsu/lsu.sv: logic LSUHADDR`

2. Create a new file in the **constraints** directory called **signals.txt**, which contains the following line
 - a. `wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/LSUHADDR 64 6`
3. Run **proberange**
 - a. `./proberange signals.txt new-probe.xdc`
4. Copy the contents of **new-probe.xdc** into **small-debug.xdc**
5. Rerun synthesis

23.1.2 Supporting Other Boards

The Arty A7 board is a relatively affordable platform compared to larger and more expensive FPGAs found on boards such as the VCU108 and VCU118. However, Wally uses nearly all the FPGA resources and is limited to a 20 MHz clock. Other FPGAs enable the possibility of multicore, larger caches, accelerators, and higher clock speeds. This section describes how to support two additional boards: the VCU108 and VCU118. The steps can be generalized to support any new board.

The VCU108 contains a very large and fast AMD Vertex UltraScale XCVU095-2FFVA2104E FPGA. Unlike the Arty A7 board, the VCU boards are targeted to research and industry as loss leaders to get engineers comfortable using expensive FPGAs. The FPGA board costs an impressive \$21,000, but AMD subsidizes the board cost down to \$7,700 to attract new customers. Similarly, the VCU118 is based around the newer UltraScale+ XCVU9P-L2FLGA2104E, which costs an eye-dropping \$59,000 on its own, but because of AMD discounts, the board costs just \$9,099.

<SIDEBAR>

A loss leader is a business strategy where a product is sold at a loss with the goal of enticing customers to buy other more expensive products. Milk, bread, and eggs are common loss leaders.

</END>

Both boards still include PMOD connectors for the flash card and USB-to-UART converter chips, but they also add 5 GiB of DDR4 DRAM and many fast peripheral devices. Five main changes are required to support the new boards:

1. Board files
2. New top-level RTL
3. Memory controller
4. Clocking and timing constraints
5. I/O pin mappings
6. Updating the device tree and ZSBL (zero stage bootloader)

The majority of commercial FPGA boards come with templates which define the FPGA type and size, pin mappings for some I/O devices, and board clocks. The VCU boards' files come with Vivado. Many of the Digilent boards are found in **addins/vivado-boards** (from the github.com/Digilent/vivado-boards repository) and were installed as part of Section 23.1.1.2.

Once the boards' files are installed, we can set a `partNumber` and `boardName` in the `cvw/fpga/Makefile`. For example, the VCU118's FPGA and `partNumber` is `xcvu9p-flga2104-2L-e` and its `boardName` is `xilinx.com:vcu118:part0:2.4`.

The VCU boards have a different enough architecture and naming convention for ports and other board connections that they require a new top-level Verilog module `fpgatopvcu.sv`. For example, the Arty A7 board uses a 256-MiB DDR3 while the VCU boards use DDR4. To use the full 5 GiB, two memory controllers are required. As of this time, only one memory controller is implemented, so only 2.5 GiB of memory is available in the current implementation. Make sure to update `cvw/config/fpga/config.vh` with the new memory range. Unlike the DDR3 MIG, the DDR4 MIG for the UltraScale and UltraScale+ takes a single input clock and can generate multiple output clocks. Consequently, the MMCM IP is not needed. As with the Arty A7 board, the memory controller typically runs at a higher clock speed than the CPU, so the AXI clock converter is still required to support this multi-clock domain design. Depending on the IP and the specific FPGA, an extra MMCM may be required for some designs as the MIG may or may not contain additional clock output ports.

To change the clock speed on the Arty A7, edit `cvw/fpga/generator/xlnx_mmcm.tcl` by replacing 20 with another frequency in MHz.

```
CONFIG.CLKOUT3_REQUESTED_OUT_FREQ {20}
```

Similarly, the VCU boards change their clock speed with `cvw/fpga/generator/xlnx_dds.tcl`.

```
CONFIG.ADDN_UI_CLKOUT1_FREQ_HZ {71}
```

Each FPGA board has unique pin mappings between the FPGA I/O and peripherals. The `cvw/fpga/constraints/` directory contains `xdc` (Xilinx design constraint) files for each board that map the I/O pins. For example, in `constraints-vcu118.xdc` the following line maps FPGA pin L31 to the top-level Verilog module input `UARTSin`.

```
set_property PACKAGE_PIN L31 [get_ports UARTSin]
```

The board's datasheet specifies how the FPGA pins are connected on the board. The VCU108 and VCU118 user guides can be found on AMD's website.

Multiple bugs in the VCU118 board exist, which break both the on-board USB to UART adaptor and the Digilent PmodSD board. To work around the UART issue, SparkFun makes the DEV-09873 (USB to serial breakout) board which happens to fit nicely into the male Pmod port. The 3.3V version is required and sits on the right most row of pins but not the bottom one (see Figure 23.3). The PmodSD board is a bigger problem. The root cause of the problem is that the level shifter is incompatible with the board. The VCU118 level shifter assumes the daughter boards provide strong pull-up resistors $< 300\ \Omega$; however, the PmodSD board has a different level shifter and uses 10 k Ω pullups. We built custom SD card boards with 300 Ω pullup resistors. Even with this change, the SD card controller cannot run faster than about 11 MHz without errors.

<SIDEBAR>

The design files for this custom SD board are located in `cvw/fpga/boards/pmodsd/`.

</END>

Finally, the device tree and ZSBL (see Section 23.1.3) need to be updated to reflect the changes in the size of memory, clock speeds, and any new peripherals. The clock needs to be updated for all devices in the tree. The VCU108 and VCU118 have their own device trees located in the `cvw/linux/device` directory. The bootloader likewise needs to know the clock frequency so it can set the correct flash card clock frequency. This is set in `cvw/tests/custom/boot/boot.c` as the macro `#FREQ` near the top of the file.

The bootloader must be recompiled with `make` and the memfile, `cvw/tests/custom/boot/bin/boot.memfile`, copied to `cvw/fpga/src/boot.mem`.

23.1.3 Wally Zero Stage Bootloader

The *zero stage bootloader* (ZSBL), also referred to simply as the bootloader, is system-dependent because it must manage peripherals such as the SD card clock. Wally's ZSBL is located in `cvw/fpga/zsbl/`. The ZSBL reads GPT partition 1 to find the location and size of the device tree, copying it into Wally's RAM at `0x87000000`. Partition 2 contains the OpenSBI bootloader and firmware called `FW_JUMP`, described in Section 22.2.4XREF. The ZSBL copies `FW_JUMP` into RAM at `0x80000000`. Next, it copies the Linux kernel (Section 22.2.5XZREF) from partition 3 to `0x80200000`. Finally, it flashes LED0 and LED1 5 times and jumps to `0x80000000` to start the OpenSBI bootloader.

23.1 Interesting Bugs

FPGAs aid ASIC verification by emulating and validating RTL functionality in a much higher-performance environment compared to simulation. Wally's FPGA implementation found numerous functional and performance bugs that the standard regression tests did not. This section highlights two of these bugs and ends with critical path analysis, showing how we improved clock speed by 33%.

23.1.1 Bug 1: AHB Bursts

The earliest version of Wally did not use AHB's burst mode to fill cache misses and instead issued separate XLEN-bit read or writes for each word in the cache line. While it did work, the performance was poor, taking over 5 minutes to boot Linux using the FPGA. When we added burst mode, Wally passed all regression tests, but the FPGA implementation failed. It did not display the OpenSBI banner and then got stuck in an infinite loop. Unfortunately, nothing obvious, such as an illegal instruction, provided a good starting point for investigation.

So, we used the ILA to records a short 16,384 clock sample of Wally's internal state. After failing to boot, we noticed Wally was frequently in the Linux trap handler. Eventually, we found an instruction during the trap handler was causing another trap into OpenSBI M-mode. This instruction generated a load-access fault but should not. We set up an ILA trigger on this instruction and `TrapM = 1` and restarted Wally to catch the first instance of this instruction faulting. Using an object dump of the Linux source code, we found the access fault's address. Because the ILA only captures 16,384 clock cycles, backtracking to the root cause may not be visible within a single ILA capture. This required reconfiguring the ILA multiple times and restarting the whole boot process repeatedly before finding the timeframe that showed a discrepancy between data written to memory and data read from memory, which led to the hardware bug described below.

The EBU relied on three main assumptions about the IFU and LSU memory requests.

1. The IFU makes a request and finishes before another request is made.
2. The LSU makes a request and finishes before another request is made.
3. Or both make a request on the same cycle.

The FPGA challenged these assumptions by showing it was possible for the LSU to make a request while the IFU already had a pending request. If the I\$ and DTLB miss at the same time, but the HPTW's first memory request to the D\$ hits, the I\$ will be granted access to the AHB. If later HPTW requests miss in the D\$, the LSU attempts to make a request while the IFU already has access to the bus, thus violating the above assumptions and interrupting the IFU's burst request. The non-burst implementation worked because only one word is transferred at a time, so it was possible for the IFU to have transferred part of the cache line and then have the EBU give the LSU bus access. Then, after the LSU completed, the IFU was again given bus access to finish its requests.

The ILA has an advanced trigger mode called trigger state machine, which allows for sequenced triggers. This allowed us to check the above assumption by first having the IFU make a memory request and then having the LSU make a request. If the trigger occurred then we knew the assumption failed and it would also provide additional processor state at the time of the failure.

The `ahb_ram` tests are variations on the `rv64gc` configuration which run the floating-point `fadd_b11-01` test. These enable or disable burst mode and change latencies between the caches and main memory to be 0, 1 or 2 cycles, which latencies are designed to specifically trigger this scenario.

23.1.2 Bug 2: Concurrent Exceptions

The Linux boot involved an M-mode timer interrupt with a concurrent instruction page fault. Because the page fault is handled in S-mode, the M-mode timer interrupt has higher priority and should be handled first; however, some really strange behavior emerged. `mcause` oscillated between `0x800...007` (machine timer interrupt) and `0x7` (store access fault), settling on `0x7` after several cycles. The IFU fetched an invalid address that was neither the `mepc` or `sepc`.

Just like the previous bug, this scenario never occurred in the regression test but happened during a Linux boot on the FPGA. The oscillating `mcause` pointed to a possible root cause.

`Committed(F/M)` indicate the IFU/LSU are processing an in-flight bus transaction. When either is high it prevents a `TrapM` and also would clear the msb of `mcause`. The LSU correctly disables bus access on `TrapM`; however, the IFU did not. In the first cycle of the trap, `mcause` was set correctly to `0x800...007`, but because the bus fetched data from an invalid region due to a branch misprediction, `CommittedF` raised which forced `InterruptM` to go low and made the trap look like an exception (`0x7`). The solution was to disable the bus access on `TrapM`.

23.1.3 Critical Path Analysis

In addition to finding functional bugs, an FPGA also assists in debugging non-functional parameters of an SoC such as power, performance, and area. Recall that the critical path is the longest combinational logic delay between two flip-flops and that it sets the maximum clock speed of a synchronous design. As the critical path decreases, the clock speed increases, boosting performance.

The first Wally version to boot into Linux on the Arty A7 FPGA only ran at 15 MHz or a critical path of $1/15 \text{ MHz} = 66.7 \text{ ns}$. By analyzing the timing paths, it was possible to increase the clock speed to 20 MHz with some simple RTL modifications.

Figure 23.11 shows a snippet of the Vivado timing report before optimization. The critical path begins at the PCF register, `pcreg/q_reg[3]/Q`, and passes through the PC+4 adder, `PCPlus4F_inferred_i_4/O[0]`, and into the HPTW. The HPTW uses `PCPlus4F` to walk the page table on a TLB miss and produce `HPTWAdr`. This greatly increases the critical path by placing this adder at the start of the path. However, the HPTW does not use `PC+2` in the same cycle the ITLB miss is detected and `PC+2` stays constant during the entire page table walk. This is called a *multicycle path*, which can be resolved in two ways. First, a register in the HPTW samples `PC+4` to prevent the path from existing. Second, we write a multipath timing constraint which specifies this specific path takes 2 cycles. The first option is easier but consumes extra hardware. It also has the advantage of carrying over to the ASIC implementation. The second option uses less hardware but is hard to maintain if the hardware is updated.

Figure 23.12 shows the impact of adding the extra register to the timing path. The starting point is no longer `pcreg` so there is less logic in the critical path to `PCSpillF`.

```
wallypipelinedsocwrapper/wallypipelinedsoc/core/ifu/pcreg/q_reg[3]/Q
wallypipelinedsocwrapper/wallypipelinedsoc/core/ifu/PCF[3]
wallypipelinedsocwrapper/wallypipelinedsoc/core/ifu/PCPlus4F_inferred_i_16/CO[3]
wallypipelinedsocwrapper/wallypipelinedsoc/core/ifu/PCPlus4F_inferred_i_16_n_0
wallypipelinedsocwrapper/wallypipelinedsoc/core/ifu/PCPlus4F_inferred_i_15/CO[3]
...
wallypipelinedsocwrapper/wallypipelinedsoc/core/ifu/PCPlus4F_inferred_i_5/CO[3]
wallypipelinedsocwrapper/wallypipelinedsoc/core/ifu/PCPlus4F_inferred_i_5_n_0
wallypipelinedsocwrapper/wallypipelinedsoc/core/ifu/PCPlus4F_inferred_i_4/O[0]
wallypipelinedsocwrapper/wallypipelinedsoc/core/ifu/Spill.spill/PCPlus4F[51]
```

```
wallypipelinedsocwrapper/wallypipelinedsoc/core/ifu/Spill.spill/PCSpillIF[51]_INST_0/O
wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/hptw.hptw/vadrmux/d0[51]
wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/hptw.hptw/vadrmux/y[51]_INST_0/O
wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/hptw.hptw/hptwwrites.HPTWAdrWriteReg/y[39]
wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/hptw.hptw/hptwwrites.HPTWAdrWriteReg/IHAdM[55]_INST_0_i_26/O
wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/hptw.hptw/hptwwrites.HPTWAdrWriteReg/IHAdM[55]_INST_0_i_26_n_0
wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/hptw.hptw/hptwwrites.HPTWAdrWriteReg/IHAdM[55]_INST_0_i_10/O
wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/hptw.hptw/hptwwrites.HPTWAdrWriteReg_n_68
wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/hptw.hptw/ITLBWriteF_inferred_i_4/O
...
```

Figure 23.11 HPTW critical path before optimization

```
wallypipelinedsocwrapper/wallypipelinedsoc/core/ifu/Spill.spill/CurrState_reg[0]/Q
wallypipelinedsocwrapper/wallypipelinedsoc/core/ifu/Spill.spill/pcplus2reg/out[0]
wallypipelinedsocwrapper/wallypipelinedsoc/core/ifu/Spill.spill/pcplus2reg/PCSpillIF[46]_INST_0/O
wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/hptw.hptw/vadrmux/d0[46]
wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/hptw.hptw/vadrmux/y[46]_INST_0/O
wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/hptw.hptw/hptwwrites.HPTWAdrWriteReg/y[34]
wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/hptw.hptw/hptwwrites.HPTWAdrWriteReg/IHAdM[55]_INST_0_i_29/O
wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/hptw.hptw/hptwwrites.HPTWAdrWriteReg/IHAdM[55]_INST_0_i_29_n_0
wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/hptw.hptw/hptwwrites.HPTWAdrWriteReg/IHAdM[55]_INST_0_i_12/O
wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/hptw.hptw/hptwwrites.HPTWAdrWriteReg/q_reg[41]_0
wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/hptw.hptw/hptwwrites.HPTWAdrWriteReg/IHAdM[55]_INST_0_i_3/O
wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/hptw.hptw/hptwwrites.HPTWAdrWriteReg/q_reg[47]_0
wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/hptw.hptw/hptwwrites.HPTWAdrWriteReg/IHAdM[55]_INST_0_i_1/O
wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/hptw.hptw/hptwwrites.HPTWAdrWriteReg/IHAdM[55]_INST_0_i_1_n_0
wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/hptw.hptw/hptwwrites.HPTWAdrWriteReg/IHAdM[26]_INST_0/O
wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/dmmu.dmmu/tlb.tlb/tlbcam/camlines[26]/keyflop/D[14]
wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/dmmu.dmmu/tlb.tlb/tlbcam/camlines[26]/keyflop/PhysicalAddress[47]_INST_0_i_177/O
wallypipelinedsocwrapper/wallypipelinedsoc/core/lsu/dmmu.dmmu/tlb.tlb/tlbcam/camlines[26]/keyflop/PhysicalAddress[47]_INST_0_i_177_n_0
```

Figure 23.12 HPTW critical path after optimization

By repeatedly iterating over the timing report and updating the RTL, we increased the FPGA's clock speed from 15 MHz to 20 MHz. The other major changes included noticing the LSU disabled AHB bus requests by ANDing NOT IgnoreRequestTLB into CacheRW if the HPTW was busy. When the DTLB misses, the memory access is ignored by asserting IgnoreRequestTLB. This eventually fed into the cachesm and drove SelfFlush, which selected CacheBusAdr. CacheBusAdr then drove into the AHB Bus, thus generating a very long critical path. We were able to refactor the code so IgnoreRequestTLB did not drive BusRW or CacheRW.

23.2 Summary

This chapter shows how to implement Wally on Xilinx FPGAs ranging from a mid-range Digilent Arty A7-100T board or a high-end AMD VCU118 FPGA evaluation board. The fpgatopArtyA7 System Verilog module instantiates the wallypipelinesoc, along with clock and reset circuitry, an AXI bridge to a DDR DRAM memory controller, an SD card controller for an on-board flash memory, and a UART-to-USB interface to communicate with a terminal on a host computer.

The Wally zero-stage boot loader (ZSBL) reads the OpenSBI bootloader, device tree, and Linux kernel from the SD card into DRAM and then jumps to the OpenSBI bootloader. The remainder of the OS boot is the same on an FPGA as in simulation.

FPGAs are invaluable tools in verification and performance analysis. Using the higher performance of FPGAs, Wally is able to execute hundreds of billions of instructions faster to find bugs the standard regression test failed to uncover in simulation. FPGAs tradeoff execution speed for internal signal visibility, making debugging harder once a bug is detected. Finally, FPGAs identified key critical path issues without the effort of going through an ASIC synthesis and place and route design flow. In summary, FPGAs are valuable tools for studying ASIC designs without the significant overhead of the ASIC design flow.